# Blockchain Lab 03 — Consensus Simulation

Szatmáry Kornélia Sára (Y5COAO)

**Repository:**
https://github.com/korneliasara/blockchain-lab03-consensus-simulation

---

# 1. Introduction

This report presents my implementation for **Blockchain Lab 03**, which focuses on simulating three core aspects of blockchain consensus:

1. **Proof of Work (PoW)**
2. **Proof of Stake (PoS) with Casper FFG-style finality**
3. **Asynchronous gossip networking with partitions and recovery**

The project was implemented in Python and organized into a `consensus` package containing modules for chain handling, PoW, PoS, networking, and a command-line interface. In addition to the implementation itself, I created experiments and test cases to verify correctness and to illustrate the behavior of each consensus mechanism.

My implementation passed:

- **57/57 local tests**
- **42/42 professor reference tests**

This confirms that the implemented functionality satisfies both my own validation suite and the externally provided specification.

---

# 2. Part A — Proof of Work

## 2.1 Objective

The goal of Part A was to implement a simplified **Proof of Work blockchain**, including:

- valid transaction application,
- deterministic block hashing,
- mining,
- target/difficulty conversion,
- retargeting,
- chain selection by cumulative work,
- and parallel nonce search.

## 2.2 Implementation summary

The PoW module supports mining blocks whose hash is below a target derived from the current difficulty. Difficulty can be adjusted over time using a retargeting rule based on observed mining intervals. The blockchain chooses the valid branch with the greater cumulative work, which models Nakamoto-style consensus.

I also implemented **parallel mining**, where multiple workers search disjoint nonce ranges for a valid solution.

## 2.3 Parallel mining demonstration

In the parallel proof-of-work demo, the same valid nonce was found while varying the worker count:

- **1 worker** → valid PoW found, elapsed time ≈ **0.2671 s**
- **2 workers** → valid PoW found, elapsed time ≈ **0.2893 s**
- **4 workers** → valid PoW found, elapsed time ≈ **0.2049 s**

This demonstrates that the implementation supports parallel search correctly and that more workers can improve search time, although timing still varies because mining remains probabilistic.

## 2.4 Mining statistics

I simulated the mining of **100 blocks** with a target interval of **1.000 s**. The resulting measurements were:

| Metric | Value |
|---|---|
| Blocks mined | 100 |
| Target interval | 1.000 s |
| Mean interval | 0.7611 s |
| Median interval | 0.5157 s |
| Standard deviation | 0.8348 s |
| Median / mean | 0.678 |
| Std. dev. / mean | 1.097 |

The histogram of block intervals shows the typical **right-skewed distribution** expected from PoW mining: many short intervals, but occasionally much longer waiting times. This reflects the probabilistic nature of hashing-based leader selection.

**Figure 1.** *Proof of Work - Distribution of 100 Block Intervals*
**[Insert image: `partA_interval_histogram.png`]**

## 2.5 Difficulty retargeting

The difficulty-adjustment experiment showed that the implemented retargeting logic correctly responds to observed mining speed. When blocks are found too quickly, difficulty increases; when they are found too slowly, difficulty decreases or remains stable according to the implemented limits.

The resulting difficulty evolution over 100 blocks confirms that the retargeting mechanism is functioning and adapting the target over time.

**Figure 2.** *Proof of Work - Difficulty Adjustment*
**[Insert image: `partA_difficulty_plot.png`]**

### 2.6 Interpretation

Overall, Part A demonstrates the essential behavior of Proof of Work:

- mining is random,
- block intervals vary significantly,
- difficulty retargeting helps stabilize long-term timing,
- and cumulative work determines the winning chain.

---

# 3. Part B — Proof of Stake / Casper FFG

### 3.1 Objective

The aim of Part B was to simulate a simplified **Casper FFG-style Proof of Stake protocol**, including:

- validators and effective balances,
- attestations,
- checkpoint justification,
- checkpoint finalization,
- slashing conditions,
- correlated slashing penalties,
- and inactivity leak behavior.

### 3.2 Finality demonstration

The finality demo shows how checkpoints become justified and then finalized over successive epochs.

Observed behavior:

- **Epoch 1:** checkpoint 1 became justified; no new checkpoint finalized yet.
- **Epoch 2:** checkpoint 2 became justified; checkpoint 1 became finalized.
- **Epoch 3:** checkpoint 3 became justified; checkpoint 2 became finalized.

This matches the core idea of Casper FFG: finality requires justified checkpoints in consecutive epochs, so finalization always lags behind justification.

### 3.3 Correlated slashing

I implemented slashing logic for conflicting validator behavior and measured the penalty per validator for different numbers of simultaneous offenders.

| Simultaneous offenders | Loss per validator (ETH) |
|---|---|
| 1 | 2.4084 |
| 5 | 8.0558 |
| 10 | 15.1462 |
| 20 | 29.4320 |
| 30 | 32.0000 |

The results show that the loss per validator rises sharply as more validators are slashed together. This models **correlated slashing**, where coordinated misbehavior is punished more severely than isolated faults. In the most extreme case shown, the loss approaches the full **32 ETH** stake.

**Figure 3.** *Casper FFG - Correlated Slashing Penalties*
**[Insert image: `partB_slashing_plot.png`]**

## 3.4 Inactivity leak

I also simulated an inactivity leak scenario, where a large set of validators becomes absent. Initially, only **24 of 64 validators** remained active, corresponding to a participation share of:

\[ 24 / 64 = 37.5\% \]

This is below the two-thirds threshold required for finality, so the chain could not finalize. Over time, however, the inactive validators lost effective stake through the inactivity leak mechanism, increasing the active validators' relative voting power.

Selected observations from the experiment:

- Epoch 1: voting share **37.50%**
- Epoch 10: **39.36%**
- Epoch 20: **43.18%**
- Epoch 40: **51.01%**
- Epoch 60: **58.79%**
- Epoch 80: **66.16%**
- Epoch 82: **66.86%**
- Epoch 83 onward: **67.21%**

Finality resumed once the effective active share crossed the required threshold. In the output, justification/finalization resumed around **epochs 82–83**, after which finalized checkpoints continued progressing normally.

This experiment illustrates the purpose of the inactivity leak: even if many validators disappear, the protocol can eventually recover liveness by gradually reducing the influence of non-participating stake.

**Figure 4.** *Casper FFG - Inactivity Leak and Finality Recovery*
**[Insert image: `partB_inactivity_plot.png`]**

### 3.5 Interpretation

Part B demonstrates that Proof of Stake consensus is not only about selecting validators, but also about ensuring **safety** and **liveness**:

- safety through slashing conflicting votes,
- liveness through finalization rules and inactivity penalties,
- and economic accountability through stake-weighted behavior.

---

## 4. Part C — Gossip Networking and Partition Recovery

### 4.1 Objective

The goal of Part C was to model a **gossip-based blockchain network**, including:

- asynchronous propagation of blocks and transactions,
- duplicate-message suppression,
- invalid-transaction rejection,
- network partitions,
- healing/reconnection,
- and chain reconciliation after divergence.

### 4.2 Network partition scenario

In the partition demo, a six-node network was split into two groups:

- **Group A:** nodes `[0, 1, 2]`
- **Group B:** nodes `[3, 4, 5]`

While partitioned, both groups continued producing blocks independently.

Before reconnection:

- Nodes in **Group A** had:
  - height = **3**
  - work = **262144**
- Nodes in **Group B** had:
  - height = **5**
  - work = **393216**

Thus, Group B had accumulated the heavier chain.

### 4.3 Reconnection and reorganization

After the network healed, all nodes exchanged information and reconciled their local views. The heavier branch won, so Group A nodes reorganized to follow Group B's longer/heavier chain.

After reconnection:

- Nodes `0`, `1`, `2`:
  - height = **5**
  - reorgs = **1**
  - reorg depth = **3**
- Nodes `3`, `4`, `5`:
  - height = **5**
  - reorgs = **0**

Final result:

- **All nodes agree:** `True`
- **Winning height:** `5`

This confirms that the implementation correctly handles:

- divergence during isolation,
- reconciliation after healing,
- adoption of the heavier chain,
- and explicit recording of reorganizations.

### 4.4 Interpretation

This part illustrates an important real-world blockchain property: temporary disagreement between nodes is normal in distributed systems, especially under partitions or delays. Consensus is restored when communication resumes and nodes compare competing histories. The heavier chain rule ensures convergence to a single canonical chain.

---

# 5. Testing and Validation

The implementation was validated using two independent test suites.

### 5.1 Local test suite

My own file `test_consensus.py` contains tests for:

- transaction validation,
- deterministic transaction IDs,
- genesis blocks,
- PoW mining and retargeting,
- parallel mining,
- validator accounting,

- justification and finalization,
- slashing,
- inactivity leak,
- gossip propagation,
- network partitions,
- reconnection,
- reorganizations,
- and asynchronous messaging.

Result:

- **57 passed in 9.99 s**

### 5.2 Professor reference tests

I also ran the professor's external reference test suite.

Result:

- **42 passed in 3.87 s**

This confirms that the project satisfies the official expected behavior.

---

# 6. Conclusion

In this lab, I implemented and tested three complementary blockchain-consensus components:

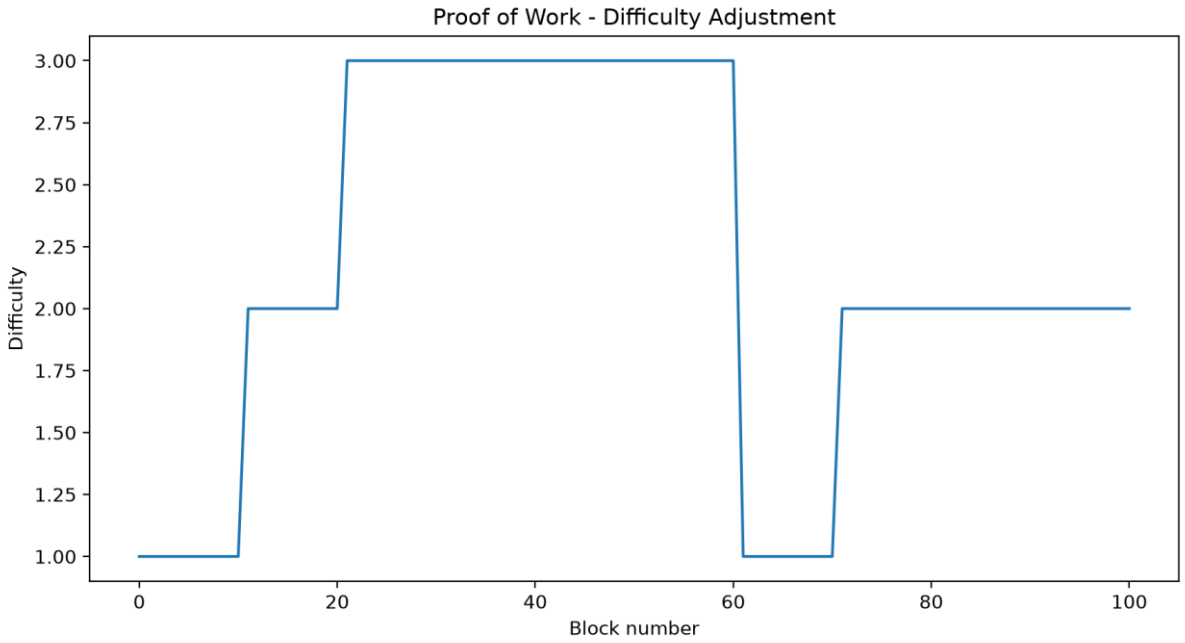
- **Proof of Work**, demonstrating probabilistic mining, difficulty retargeting, and cumulative-work fork choice;
- **Proof of Stake with Casper FFG**, demonstrating checkpoint finality, slashing, and inactivity-leak-based liveness recovery;
- **Asynchronous gossip networking**, demonstrating transaction/block propagation, partition handling, and consensus restoration after reconnection.

The experimental outputs match the theoretical expectations:

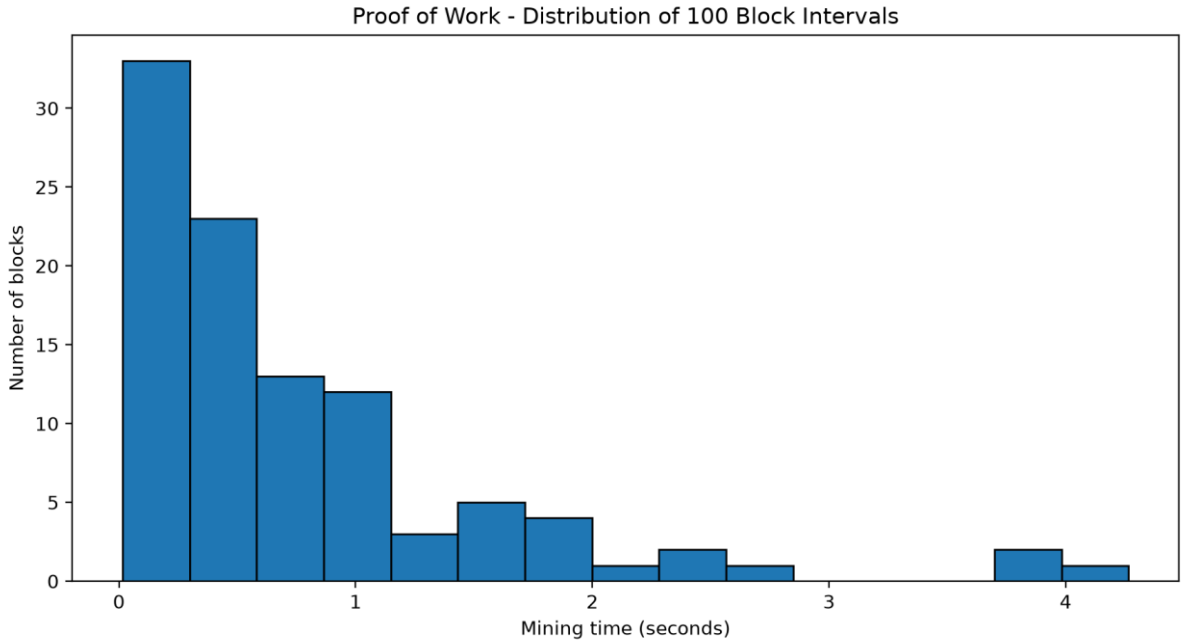
- PoW block times are variable and approximately exponentially distributed;
- difficulty retargeting adapts to timing changes;
- Casper FFG finality emerges only after consecutive justified checkpoints;
- correlated misbehavior is penalized more strongly;
- inactivity leak restores finality after major validator absence;
- network partitions create temporary divergence, but healing restores a single canonical chain.

The fact that both my own tests and the professor's reference tests pass indicates that the implementation is functionally correct and consistent with the lab specification.

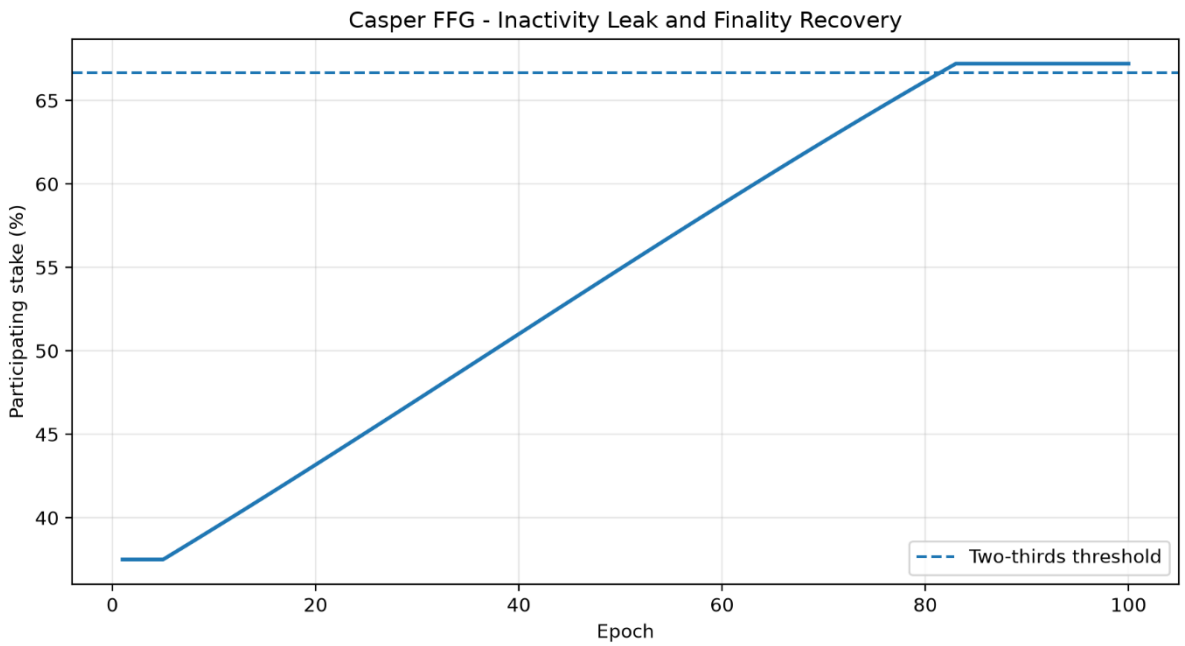
# Figures:



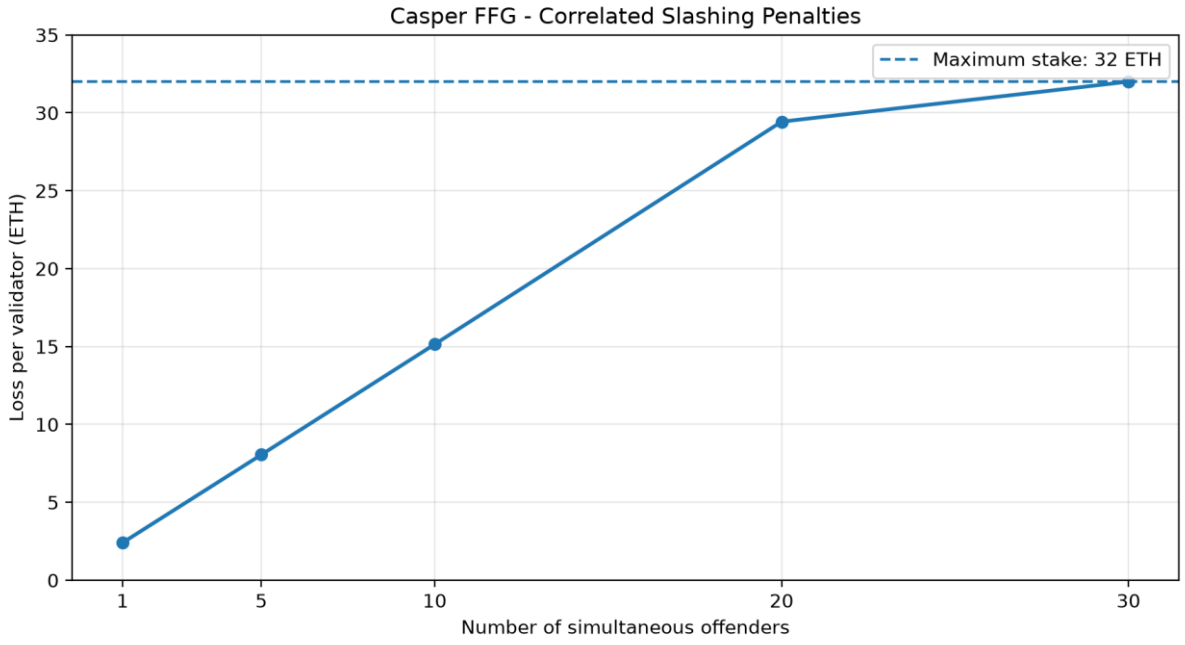

*1. Figure part A difficulty plot*




*2. Figure part A intervall histogram*




*3. Figure part B inactivity plot*




*4. Figure part B slashing plot*